

E3 - Implementing advanced graphics with PSOC™ Edge

Customer training workshop (CTW)

July 2026

Public

Table of contents

1 **Prerequisites** **2**

2 Graphics subsystem overview 5

3 Graphics subsystem labs 16

4 Graphics performance and power 20

5 Graphics performance and power lab 25

Copyright © Infineon Technologies AG 2026. All rights reserved.

Prerequisites

Recommended training prerequisites

- Introduction to PSOC™ Edge and ModusToolbox™
  - For an introduction to PSOC™ MCUs, including a getting started guide to ModusToolbox™, go to https://www.infineon.com/product-information/psocdeveloper
  - For PSOC™ Edge training, from beginner tutorials to advanced training, go to PSOC™ Edge E84 Training Collection



Copyright © Infineon Technologies AG 2026. All rights reserved.

Prerequisites: Tools

- Development tools for all labs (see training manual for links)
  - ModusToolbox™ v3.7
  - Eclipse IDE for ModusToolbox™ v2025.8.0
  - Edge protect security suite v1.6
  - ModusToolbox™ programming tools v1.7.0
  - KIT_PSE84_EVAL_EPC2 BSP v1.0.0
  - YUV player application v2.5.0
  - EEZ studio 0.24.0
  - Terminal emulator
  - KIT_PSE84_EVAL
    - Including Waveshare 4.3-inch Raspberri Pi DSI 800x480 display



Note: Other tool versions and hardware can be used with minimal or no modifications to this training manual.

Copyright © Infineon Technologies AG 2026. All rights reserved.

Table of contents

1 Prerequisites 2

2 **Graphics subsystem overview** **5**

3 Graphics subsystem labs 16

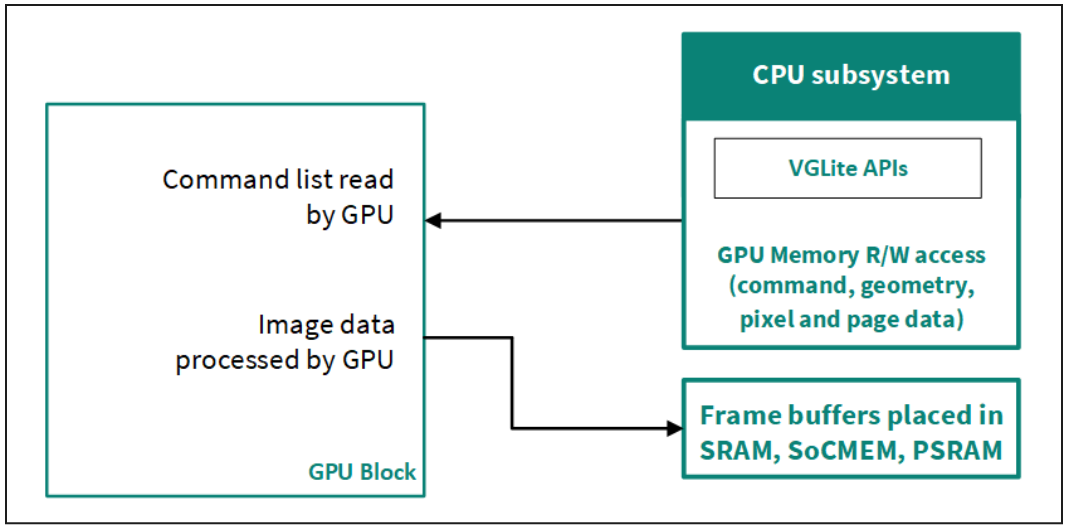
4 Graphics performance and power 20

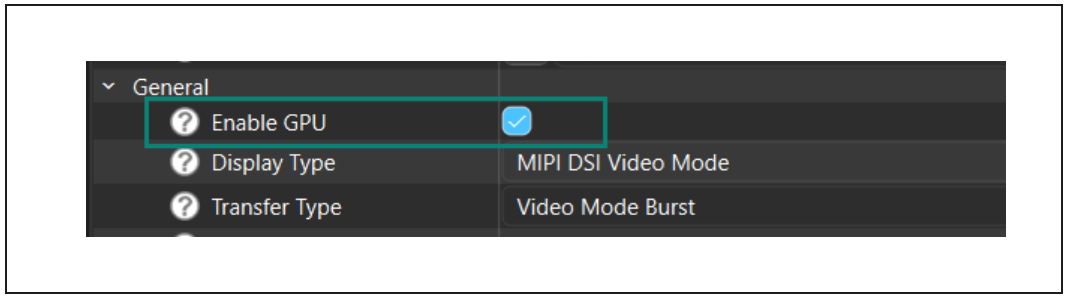
5 Graphics performance and power lab 25

Copyright © Infineon Technologies AG 2026. All rights reserved.

Graphics processing unit (GPU)

- Standalone block without any direct interfacing with other components
- Frame buffer can be stored in SRAM, SoCMEM, or PSRAM for writing the rendered pixel data
- Enabling/Disabling can be done through the device configurator personality
- For dynamically turning on and off, PDL APIs can be used
- VGLite APIs to access the GPU for vector rendering

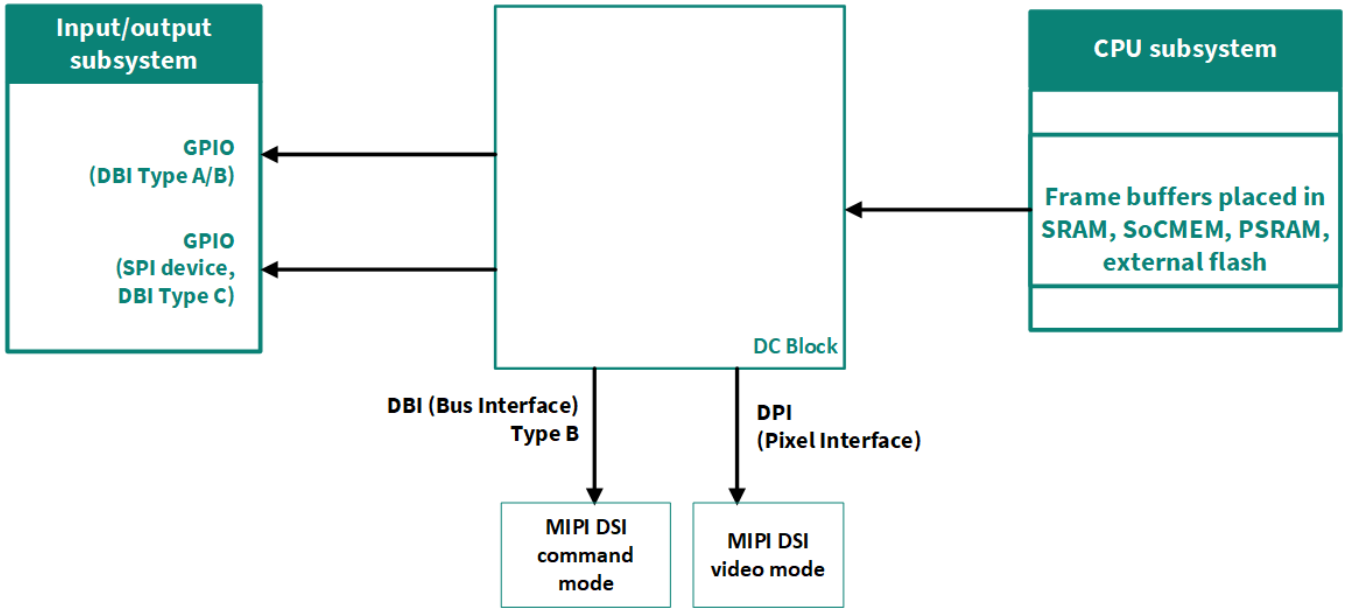




Copyright © Infineon Technologies AG 2026. All rights reserved.

The Display controller

- Reads pixel data from the frame buffers placed in interfaced memory
- Pixel data can be generated by GPU/CPU
- Various output options

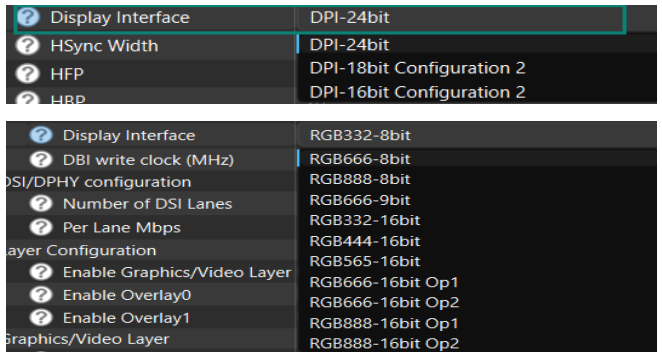


Copyright © Infineon Technologies AG 2026. All rights reserved.

Configuring display controller for the required display

Display interface

- DPI-based display interfaces for DSI video mode display panels
- DBI-based display interfaces for DSI command mode display panels



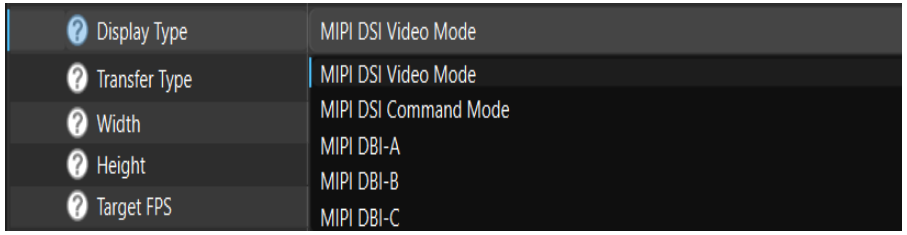
Display interface

Display resolution and type

- Vertical and horizontal resolution
- When using RGB565 (16-bit) color format, the horizontal resolution (width) needs to be aligned to 128 bytes to match the DC stride alignment
- The display types are supported, as shown in the figure



Display resolution



Display type

Copyright © Infineon Technologies AG 2026. All rights reserved.

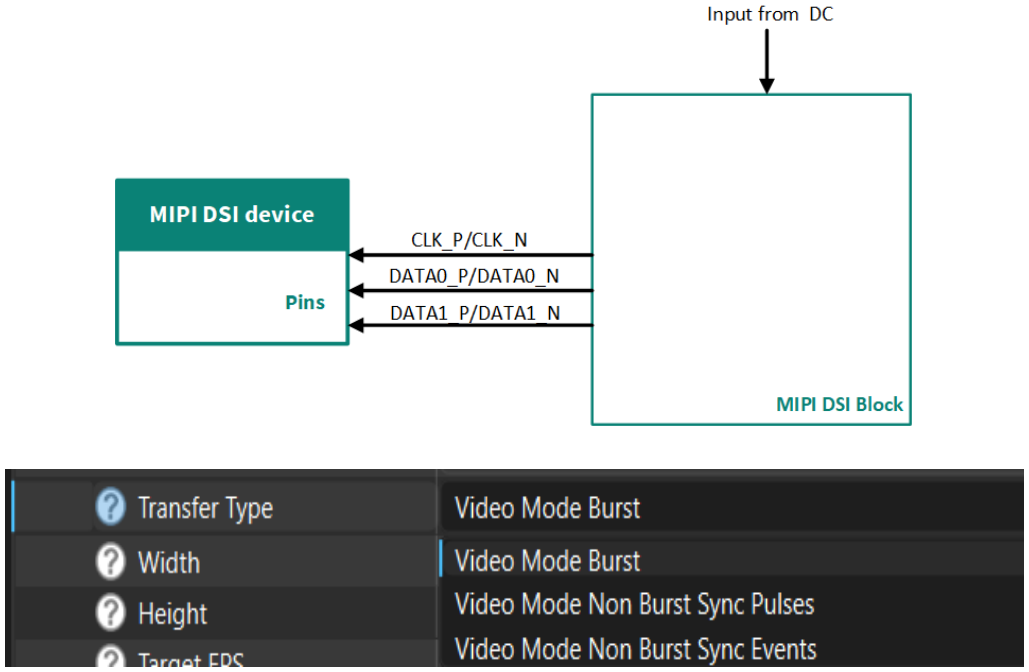
Run-length adaptive dithering (RLAD) decoder

- RLAD is a proprietary image compression format
- PSOC™ Edge supports a hardware RLAD decoder as part of the Display controller
- Supports lossy and lossless decompressions according to TGA specs
- Image is encoded offline and decoded on-the-fly
- Use case
  - Store the static image in a compressed format and decompress while rendering
  - Reduces static memory consumption
  - Example: Smartwatch face with moving hands, date, and other information as a dynamic foreground. Smart watch background is stored in compressed form and decoded on the fly
- Lab exercise 2 covers the usage of this feature

Copyright © Infineon Technologies AG 2026. All rights reserved.

Mobile industry processor interface display serial interface (MIPI DSI) host controller and DSI video mode

Transfer type	Description
Video mode burst	Active pixels are sent in high-speed bursts, with blanking or low-power idle periods in between to reduce the high-speed duty cycle. This requires a DSI payload pixel FIFO capable of storing a full line of active pixel data
Video mode non burst sync pulses	HSYNC and VSYNC are transmitted as pulse packets, while blanking and active periods are streamed continuously, aligning DC interface pixel bandwidth with the DSI link. This reduces memory requirements, as only one video packet, not a full line of pixel data, needs to be stored
Video mode non burst sync events	HSYNC/VSYNC are indicated by "start" and "end" events, with blanking defined by the time between them instead of explicit sync-pulse packets. The link streams continuously in high-speed during active video, reducing control overhead. Continuous clock-lane operation requires precise timing for porch and sync programming to define blanking intervals



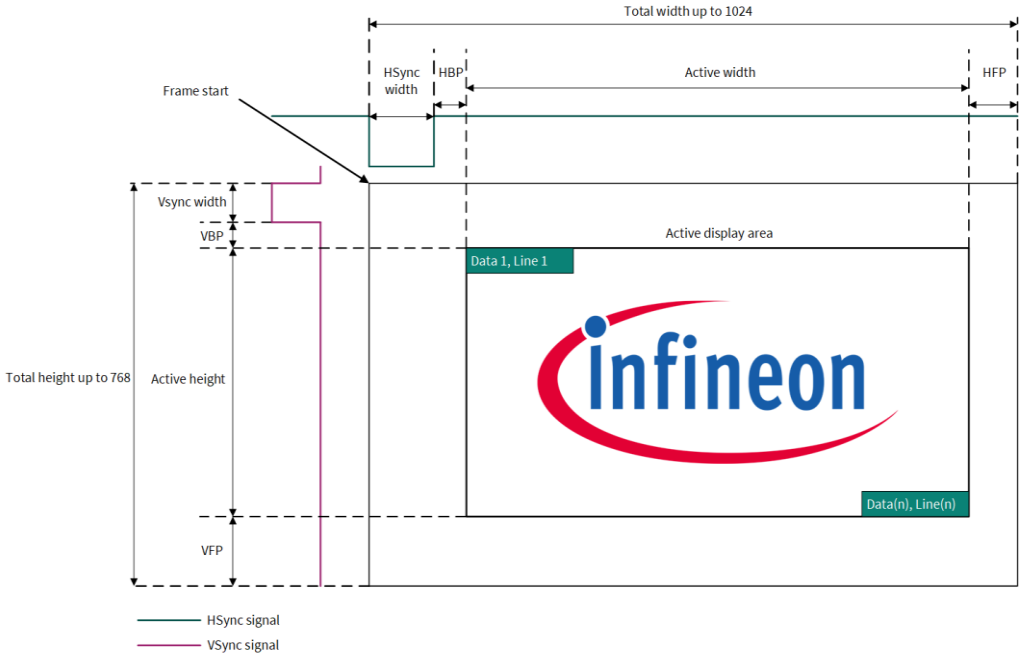
Supported transfer types for DSI video mode

Copyright © Infineon Technologies AG 2026. All rights reserved.

DSI video timings

- Timing parameters are specific to the display being interfaced
- These are typically provided in the display panel datasheet

Timing parameter	Description
Hsync width	Duration of the horizontal sync pulse, signalling the start of a new line.
HFP	Idle period after the active pixels before the horizontal sync pulse. Used by the display to prepare for the next line.
HBP	Idle period after the horizontal sync pulse before starting to send active data pixels.
Vsync width	Number of lines during the active vertical sync pulse.
VFP	Number of idle lines after completing active frame lines before the vertical sync pulse.
VBP	Number of idle lines after the VSync pulse before starting the next frame.



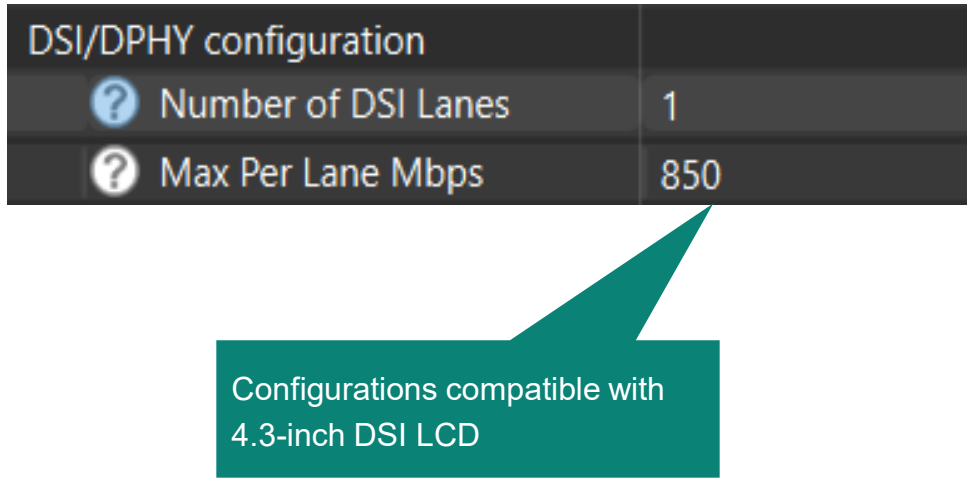
HSync Width	10
HFP	210
HBP	20
VSync Width	5
VFP	20
VBP	20

Timing parameters for 4.3-inch DSI LCD

Copyright © Infineon Technologies AG 2026. All rights reserved.

DSI video mode

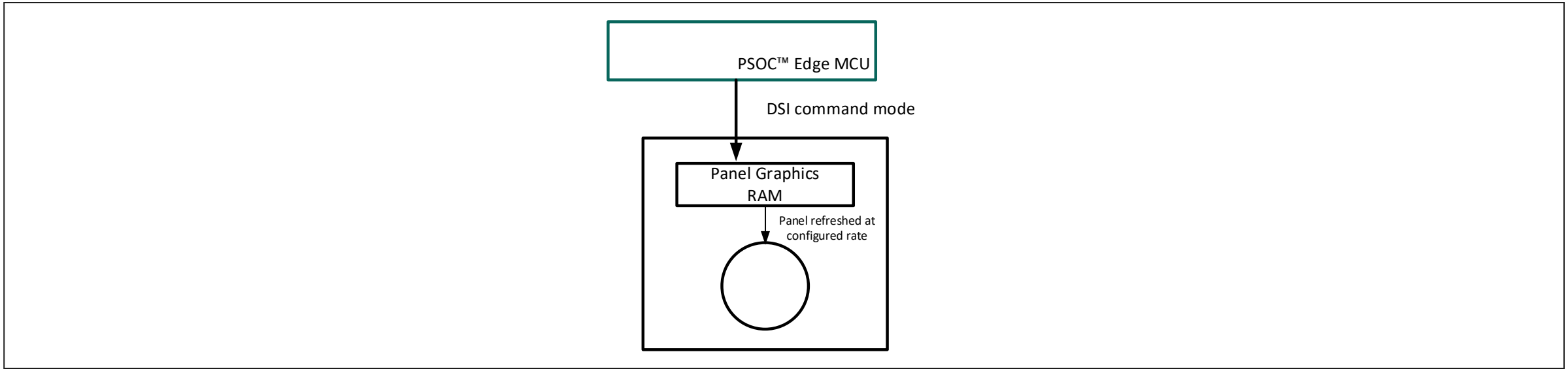
- PSOC™ Edge supports the following configurations:
  - Number of DSI lanes = 2 (Max)
  - Per lane Mbps = Max 1500 Mbps
- Number of lanes and Max per lane Mbps must be compatible with the interfaced display



Copyright © Infineon Technologies AG 2026. All rights reserved.

DSI command mode

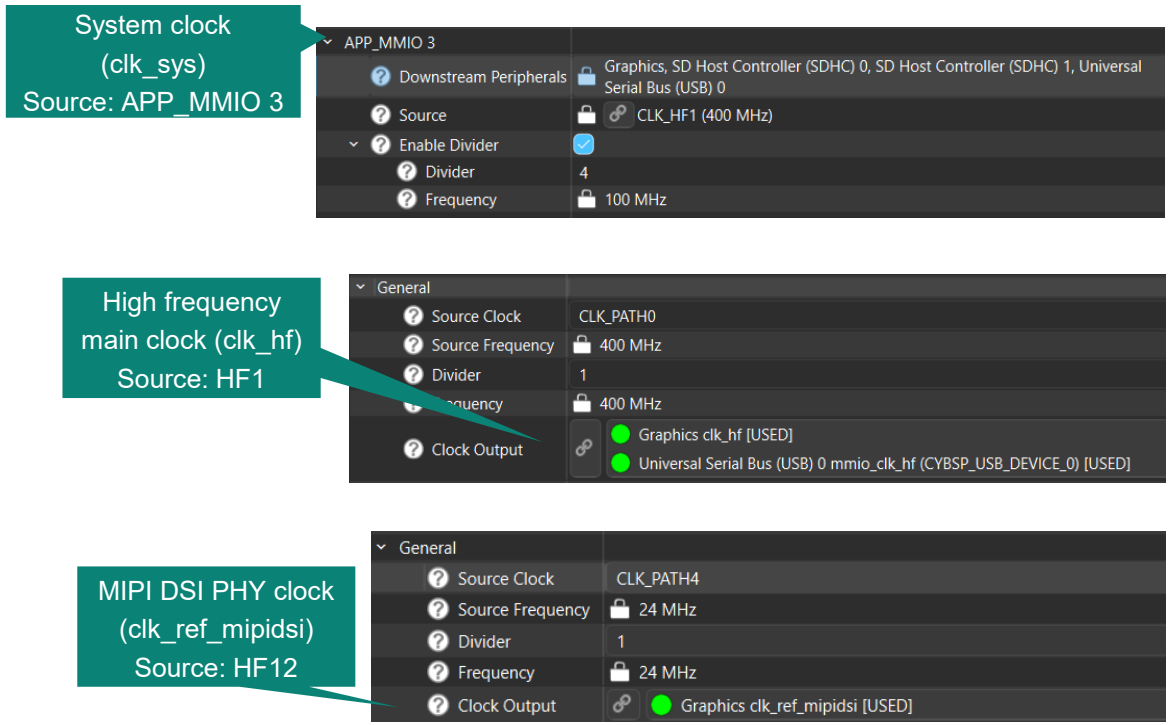
- Display panel refresh driven by commands and data via DSI link
- No continuous pixel stream as in video mode
- Pixel data written into display panel's Graphics RAM (GRAM) using short and long packets. The panel then refreshes the image stored in the GRAM on to the screen
- Supports asynchronous updates for modified regions only thereby reducing bus traffic and power consumption
- Suitable applications: Smartwatches, wearables, and so on



Copyright © Infineon Technologies AG 2026. All rights reserved.

Clocking

- The graphics subsystem has the following clock domains:
  - System clock (clk_sys)
  - High-frequency main clock (clk_hf)
  - MIPI DSI host clock (clk_ref_mipidsi)
- Used to derive internal clocks



Copyright © Infineon Technologies AG 2026. All rights reserved.

Interrupts

- The graphics subsystem has three independent interrupts as follows:
- **GPU Interrupt (interrupt_gpu)**
  - Source: AXI error responses and other core events handled by the VGLite driver
  - Usage: VGLite IRQ handling on “Composition complete” event
- **Display controller interrupt (interrupt_dc)**
  - Source: Frame complete event, DBI configuration error, RLAD decompression error
  - Usage: Enable the DC interrupt to detect the “frame complete” event, allowing the application to schedule the next frame
- **MIPI DSI Host controller interrupt (interrupt_mipidsi)**
  - Source: Tearing effect (TE) interrupt in command mode, Command Tx error in video mode
  - Usage: TE interrupt is used for partial refresh in command mode, to detect DSI link issues

Copyright © Infineon Technologies AG 2026. All rights reserved.

Table of contents

1 Prerequisites 2

2 Graphics subsystem overview 5

3 **Graphics subsystem labs** **16**

4 Graphics performance and power 20

5 Graphics performance and power lab 25

Copyright © Infineon Technologies AG 2026. All rights reserved.

Lab 1: Debugging graphics applications

What will you learn ?

- A common debugging technique with ModusToolbox™ on PSOC™ Edge
- How to check status of various GFXSS registers at runtime

Lab 2: Using RLAD hardware decompression

What will you learn ?

- How to use the RLAD decoder available in display controller of PSOC™ Edge
- Compare the reduction in static memory due to various compressions

Lab 3: EEZ Studio and LVGL I/O

What will you learn ?

- How to use EEZ Studio application to design a simple GUI
- How to integrate the EEZ Studio/LVGL Gui into a PSOC™ Edge project and use with the PSOC™ Edge EVK

Table of contents

1 Prerequisites 2

2 Graphics subsystem overview 5

3 Graphics subsystem labs 16

4 **Graphics performance and power** **20**

5 Graphics performance and power lab 25

Copyright © Infineon Technologies AG 2026. All rights reserved.

Clock setup based on required FPS for video mode displays

- **Required FPS** = Frame rate [Hz]
- **Total horizontal period**

  $H_{TOTAL} = \text{HSync width} + \text{HBP} + \text{Display active width} + \text{HFP [in pixels]}$
- **Total vertical period**

  $V_{TOTAL} = \text{VSync width} + \text{VBP} + \text{Display active height} + \text{VFP [in lines]}$
- **Pixel clock and DSI clock**

  $\text{PCLK} = H_{TOTAL} \times V_{TOTAL} \times \text{Frame rate [in Hz]}$

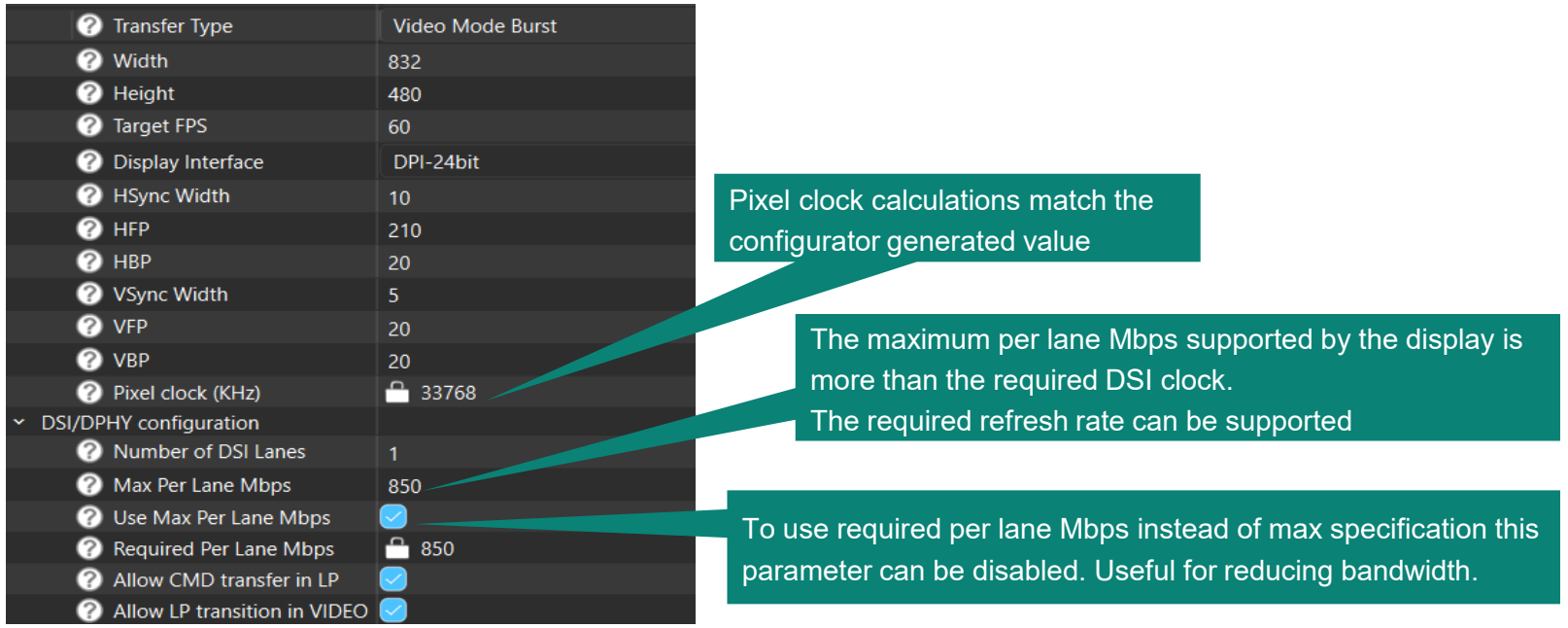
  $\text{DSI clock} = (\text{PCLK} \times \text{Bit per pixel in display interface}) / (\text{\# DSI lanes}) \text{ [in Mbps]}$

Note: Timing parameter descriptions discussed in slide-9.

Copyright © Infineon Technologies AG 2026. All rights reserved.

Pixel and DSI clock calculations for 4.3-inch DSI LCD

Parameter	Value	Calculation
H_TOTAL	1072	10 + 210 + 20 + 832
V_TOTAL	525	5 +20 +20 + 480
Pixel clock (kHz)	33,768	(1072 x 525 x 60) / 1000
DSI clock (Mbps)	~810	(33,768,000 x 24) / 1E6



Copyright © Infineon Technologies AG 2026. All rights reserved.

Clock setup for DSI command mode display

- The frame rate of a command mode display is governed by the DBI clock which depends on the lane byte clock

  $$\text{Lane byte clock} = (\text{Per lane Mbps} / 8)\ [\text{in Hz}]$$

  $$\text{DBI clock} = (\text{Lane byte clock} / 4)\ [\text{in Hz}]$$

- The DBI clock is generated from the graphics clock (clk_hf). Internal divider is set automatically to achieve DBI clock frequency derived above
- Frame data is transferred in slices of 64 KB

  $$\text{Total slices} = (\text{Width} * \text{Height} * \text{bits per pixel}) / 64\text{KB}$$

- Transfer time per 64 KB block

  $$\text{T_block_64} = (64 * 1024) / (\text{DBI clock} * \text{Bytes per clock})\ [\text{in s}]$$

- Transfer time per frame

  $$\text{T_frame} = \text{Total slices} / \text{T_block_64}\ [\text{in s}]$$

- Using these equation we can estimate the frame rate of the command mode display

Copyright © Infineon Technologies AG 2026. All rights reserved.

Performance improvement techniques

Efficient usage of GPU

- Usage of GPU improves the FPS and also reduces the CPU usage as compared to CPU rendered graphics
- Use pre-multiplied images with alpha component. Alpha pre-multiply operation is not supported by GPU and is offloaded to CPU
- Prefer color formats which avoid conversion such as RGB565 (For UI) and ARGB8888 (For blend effects)

Memory strategy

- Use double buffering with full resolution buffers
- Place the frame buffers in fast on-chip memory (SoCMEM) to reduce read/write latency for DC and GPU/CPU respectively
- Drive the CLK_HF2 (source clock for SoCMEM) at 300 MHz (max spec)
- Use the non-cacheable region of SoCMEM (gfx_mem) for frame buffers
- Align the stride and base address of each frame buffer to a 128 bytes boundary to match AXI burst sizes and cache lines
- Place performance-critical routines (e.g., ISR handlers, blit/dispatch glue) in ITCM

Copyright © Infineon Technologies AG 2026. All rights reserved.

Table of contents

1 Prerequisites 2

2 Graphics subsystem overview 5

3 Graphics subsystem labs 16

4 Graphics performance and power 20

5 **Graphics performance and power lab** **25**

Copyright © Infineon Technologies AG 2026. All rights reserved.

Lab 4: Observe FPS/CPU usage with and without GPU

What will you learn ?

- How to enable/disable GPU in LVGL based applications
- Compare the FPS and CPU usage in GPU enabled vs disabled scenarios

Power optimizations in graphics applications

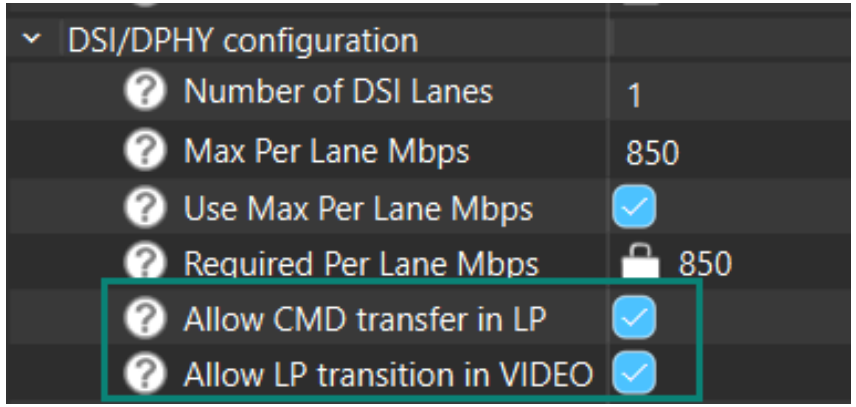
- Low-power graphics are critical for wearable applications like a smart watch
- Some common power optimizations for graphics applications are as follows:
  - Color format with low bits-per-pixel (for example, 16-bit RGB565)
  - Avoid buffer redraw when no content is changed
  - For smaller UI updates (e.g. step count in a smartwatch) use CPU rendering in place of GPU rendering
  - Delay the refresh cycles and enter the system Deep Sleep in between two refreshes
  - Use DSI ULPM mode for the clock and data lanes. Effective with command mode displays
  - Reduce screen brightness based on inactivity

Copyright © Infineon Technologies AG 2026. All rights reserved.

Low power features of MIPI DSI

– Device configurator personalities to enable power conservation features of DSI

Flag	Description
Allow CMD transfers in LP	Send short DCS commands using LP DSI packets. Reduces dynamic power consumption.
Allow LP transition in VIDEO	Transition the link from HS to LP mode when no active video packets are being sent. Reduces average link power. This feature is not supported by display which require continuous clock.



Copyright © Infineon Technologies AG 2026. All rights reserved.

Revision history

Document revision	Date	Description of change
**	2026-04-22	Initial release.
*A	2026-07-16	Updated speaker notes.

Copyright © Infineon Technologies AG 2026. All rights reserved.


Infineon